

# Conversiono V1 MVP

## Architecture Overview

AI Behavioral Intelligence for E-Commerce

Complete Technical Reference

---

7 Epics | 88 Stories | 10 Sprints | V1 Complete

FastAPI | Next.js 14 | PostgreSQL | Redis | XGBoost | SHAP

April 2026

13 Chapters | 6 Database Tables | 27 API Endpoints | 104 Tests | Full Platform Reference

# Table of Contents

---

|           |                       |
|-----------|-----------------------|
| <b>1</b>  | Platform Overview     |
| <b>2</b>  | Monorepo Structure    |
| <b>3</b>  | Backend Architecture  |
| <b>4</b>  | Database Schema       |
| <b>5</b>  | Authentication System |
| <b>6</b>  | ML Pipeline           |
| <b>7</b>  | JavaScript SDK        |
| <b>8</b>  | Dashboard Frontend    |
| <b>9</b>  | API Reference         |
| <b>10</b> | Security & GDPR       |
| <b>11</b> | Deployment            |
| <b>12</b> | What V2 Looks Like    |
| <b>13</b> | Testing               |

# Chapter 1

## Platform Overview

---

### 1.1 What Is Conversiono?

---

Conversiono is an AI-powered behavioral intelligence platform purpose-built for e-commerce. It captures real-time user behavior on merchant storefronts, applies machine learning models to detect purchase intent, friction, and abandonment risk, then delivers targeted interventions (discount popups, urgency banners, exit-intent offers) to maximize conversion rates.

Unlike traditional A/B testing tools that require manual hypothesis creation, Conversiono autonomously identifies behavioral patterns and selects optimal interventions per visitor segment. The platform combines behavioral event streaming, ensemble ML scoring, SHAP-based explainability, and a merchant-facing dashboard into a single integrated product.

### 1.2 V1 MVP Scope

---

The V1 MVP encompasses 7 completed epics (out of 10 total planned), delivering the core platform end-to-end. The remaining 3 epics (8-10) are reserved for V2.

#### Completed Epics (V1)

1. Epic 1 - Data Foundation & ML Baseline (CONV-01 to CONV-09): Raw data loading, feature extraction, baseline logistic regression models, evaluation framework.
2. Epic 2 - ML Ensemble & SHAP Explainability (CONV-10 to CONV-18): XGBoost models, ensemble scoring, SHAP explanations, intervention recommendation engine.
3. Epic 3 - JavaScript SDK (CONV-19 to CONV-27): Browser event collectors, session management, batched transport, UMD/ESM builds.
4. Epic 4 - Backend API Complete (CONV-38 to CONV-46): FastAPI application, experiment CRUD, intervention engine, cohort analytics, WebSocket, merchant

management.

5. Epic 5 - Merchant Dashboard v1 (CONV-51 to CONV-55): Next.js dashboard with overview, sessions, analytics, experiments, interventions, and settings pages.
6. Epic 6 - Security & GDPR (CONV-56 to CONV-64): JWT authentication, bcrypt passwords, security headers, rate limiting, audit logging, GDPR consent/export/delete.
7. Epic 7 - Dashboard Polish & QA (CONV-65 to CONV-70): Humanized UX, loading states, error boundaries, responsive design, accessibility compliance.

## 1.3 Technology Stack Summary

### Backend

- FastAPI 0.115+ with async/await throughout
- PostgreSQL 16 with UUID primary keys and Row-Level Security
- Redis 7 for caching, rate limiting, and session store
- SQLAlchemy 2.0 ORM with Alembic migrations
- Pydantic v2 for request/response validation

### Frontend

- Next.js 14 with App Router and server components
- React 18 with TypeScript 5
- Tailwind CSS v4 for styling
- Recharts for data visualization
- Lucide React for iconography

### ML Pipeline

- scikit-learn for baseline models (Logistic Regression)
- XGBoost for gradient-boosted ensemble models
- SHAP for model explainability and feature importance
- Pandas + NumPy for data processing

### Infrastructure

- Docker Compose for local development and deployment
- GitHub Actions for CI/CD pipeline
- Alembic for database schema migrations

### Design Philosophy

Conversiono follows a monorepo architecture with clear separation of concerns.

Each subsystem (backend, frontend, ML, SDK) is independently testable but shares a unified deployment pipeline via Docker Compose.

# Chapter 2

## Monorepo Structure

---

### 2.1 Top-Level Directory Layout

---

The Conversiono repository is organized as a monorepo with five primary directories, each owning a distinct concern. This structure allows independent development and testing of each subsystem while sharing configuration and deployment at the root level.

```
conversiono/  
|-- backend/      # FastAPI application  
|-- frontend/     # Next.js 14 dashboard  
|-- ml/           # ML pipeline & models  
|-- sdk/          # JavaScript SDK  
|-- docs/         # Documentation & PDF generators  
|-- docker-compose.yml  
|-- start.sh      # One-command launcher  
|-- .github/      # CI/CD workflows  
|-- README.md
```

### 2.2 Backend Directory

---

```
backend/
|-- app/
|   |-- main.py           # FastAPI app factory, middleware
|   |-- config.py         # Settings via pydantic-settings
|   |-- database.py       # SQLAlchemy engine, session
|   |-- models.py         # ORM table definitions
|   |-- schemas.py        # Pydantic request/response models
|   |-- auth.py           # JWT + SDK key authentication
|   |-- routers/
|       |-- sdk.py        # SDK event ingestion
|       |-- dashboard.py  # Dashboard data endpoints
|       |-- experiments.py # Experiment CRUD
|       |-- interventions.py # Intervention management
|       |-- analytics.py  # Cohort analytics
|       |-- merchants.py  # Merchant management
|       |-- auth.py       # Auth endpoints
|       |-- ws.py         # WebSocket real-time
|   |-- middleware/
|       |-- rate_limit.py # Redis-backed rate limiting
|       |-- security.py   # Security headers
|       |-- audit.py      # Audit logging
|-- tests/                # 104 pytest tests
|-- alembic/              # Database migrations
|-- requirements.txt
|-- Dockerfile
```

The backend follows a layered architecture: routers handle HTTP concerns, service logic lives in router functions (kept thin for V1), and the ORM layer manages persistence. Middleware is applied globally via the FastAPI app factory in main.py.

## 2.3 Frontend Directory

```
frontend/
|-- src/
|   |-- app/
|   |   |-- layout.tsx      # Root layout with Sidebar
|   |   |-- page.tsx        # Auth guard redirect
|   |   |-- login/page.tsx  # Login page
|   |   |-- dashboard/
|   |       |-- page.tsx     # Overview page
|   |       |-- sessions/    # Session explorer
|   |       |-- analytics/   # Analytics charts
|   |       |-- experiments/ # Experiment manager
|   |       |-- interventions/ # Intervention results
|   |       |-- settings/    # Account settings
|   |-- components/
|   |   |-- Sidebar.tsx     # Navigation sidebar
|   |   |-- DashboardShell.tsx # Layout wrapper
|   |   |-- StatCard.tsx    # Metric cards
|   |   |-- charts/         # Recharts wrappers
|   |-- lib/
|   |   |-- api.ts          # Axios API client
|   |   |-- hooks/          # Custom React hooks
|-- tailwind.config.ts
|-- next.config.js
|-- Dockerfile
```

## 2.4 ML Directory



```
ml/
|-- data/           # Raw + processed datasets
|-- models/         # Trained model artifacts
|-- src/
|   |-- data_loader.py  # CSV/parquet data loading
|   |-- features.py     # Feature extraction pipeline
|   |-- train_baseline.py # Logistic regression baseline
|   |-- train_xgboost.py # XGBoost ensemble training
|   |-- ensemble.py     # Ensemble scoring engine
|   |-- shap_explain.py  # SHAP explainability
|   |-- intervention.py  # Intervention recommendation
|-- tests/
|-- requirements.txt
```

## 2.5 SDK Directory

```
sdk/
|-- src/
|   |-- index.ts        # Main SDK entry point
|   |-- collectors/     # Event collectors (click, scroll, mouse)
|   |-- session.ts      # Session management
|   |-- queue.ts        # Event queue with batching
|   |-- transport.ts    # HTTP transport layer
|-- dist/               # Built UMD + ESM bundles
|-- rollup.config.js    # Build configuration
|-- package.json
```

## 2.6 Docs Directory

The docs/ directory contains PDF generator scripts for each epic and this architecture overview document. Each generator is a standalone Python script that uses the fpdf2 library to produce course-style documentation PDFs.

# Chapter 3

## Backend Architecture

---

### 3.1 FastAPI Application Factory

---

The backend is built on FastAPI, a modern async Python web framework. The application is constructed in `main.py` using the factory pattern: an app instance is created, middleware is attached, and routers are included with their respective prefixes.

```
# main.py - Simplified app factory
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(title='Conversiono API', version='1.0.0')

# Middleware stack (order matters - outermost first)
app.add_middleware(CORSMiddleware, ...)
app.add_middleware(SecurityHeadersMiddleware)
app.add_middleware(RateLimitMiddleware)
app.add_middleware(AuditLogMiddleware)

# Router registration
app.include_router(auth_router, prefix='/api/auth')
app.include_router(sdk_router, prefix='/api/sdk')
app.include_router/dashboard_router, prefix='/api/dashboard')
app.include_router(experiments_router, prefix='/api/experiments')
app.include_router(interventions_router, prefix='/api/interventions')
app.include_router(analytics_router, prefix='/api/analytics')
app.include_router(merchants_router, prefix='/api/merchants')
app.include_router(ws_router, prefix='/api/ws')
```

## 3.2 Router Architecture

Each router module owns a domain of the API. Routers are thin controllers that validate input via Pydantic schemas, call the database layer, and return structured responses. Authentication dependencies are injected per-route using FastAPI's Depends() system.

### Router Summary

| Router | Prefix    | Endpoints | Auth     | Purpose                            |
|--------|-----------|-----------|----------|------------------------------------|
| auth   | /api/auth | 4         | None/JWT | Registration, login, token refresh |
| sdk    | /api/sdk  | 4         | SDK Key  | Event ingestion, session create    |

|               |                    |   |     |                              |
|---------------|--------------------|---|-----|------------------------------|
| dashboard     | /api/dashboard     | 4 | JWT | Overview stats, session list |
| experiments   | /api/experiments   | 5 | JWT | Experiment CRUD + results    |
| interventions | /api/interventions | 3 | JWT | Intervention management      |
| analytics     | /api/analytics     | 2 | JWT | Cohort + funnel analytics    |
| merchants     | /api/merchants     | 4 | JWT | Merchant profile CRUD        |
| ws            | /api/ws            | 1 | JWT | Real-time WebSocket stream   |

### 3.3 Middleware Stack

Middleware is applied in a specific order to ensure correct request processing. Each middleware wraps the next, forming a pipeline that every request passes through.

#### CORS Middleware

Configured to allow the frontend origin (localhost:3000 in development) with credentials. Allows GET, POST, PUT, DELETE, OPTIONS methods. Exposes Authorization header for JWT flow.

#### Security Headers Middleware

Adds protective HTTP headers to every response: X-Content-Type-Options: nosniff, X-Frame-Options: DENY, X-XSS-Protection: 1; mode=block, Strict-Transport-Security, Content-Security-Policy with restrictive defaults, and Referrer-Policy: strict-origin-when-cross-origin.

#### Rate Limiting Middleware

Redis-backed sliding window rate limiter. Different tiers for different endpoint groups: SDK ingestion endpoints allow higher throughput (1000 req/min) while auth endpoints are more restrictive (20 req/min for login to prevent brute-force). Returns 429 Too Many Requests with Retry-After header when limits are exceeded.

#### Audit Logging Middleware

Records every state-changing request (POST, PUT, DELETE) to an audit log. Captures timestamp, user ID (from JWT), IP address, endpoint path, method, and response status. Used for GDPR compliance and security forensics.

## 3.4 Error Handling

The application uses structured error responses with consistent JSON format. Custom exception handlers catch validation errors (422), authentication failures (401), authorization failures (403), not found (404), and internal server errors (500). Each returns a response with 'detail' field containing a human-readable message.

```
# Error response format
{
  "detail": "Experiment not found",
  "status_code": 404,
  "error_type": "not_found"
}
```

# Chapter 4

## Database Schema

### 4.1 Schema Overview

Conversiono uses PostgreSQL 16 with 6 core tables. All tables use UUID primary keys generated server-side for global uniqueness. Row-Level Security (RLS) policies ensure merchants can only access their own data. Timestamps use timezone-aware UTC throughout.

### 4.2 merchants Table

Stores merchant accounts. Each merchant has a unique SDK key for browser-side authentication and a hashed password for dashboard login.

| Column          | Type         | Constraints       | Description        |
|-----------------|--------------|-------------------|--------------------|
| id              | UUID         | PK, default uuid4 | Unique merchant ID |
| email           | VARCHAR(255) | UNIQUE, NOT NULL  | Login email        |
| company_name    | VARCHAR(255) | NOT NULL          | Business name      |
| hashed_password | VARCHAR(255) | NOT NULL          | bcrypt hash        |
| sdk_key         | VARCHAR(64)  | UNIQUE, NOT NULL  | SDK auth key       |
| is_active       | BOOLEAN      | DEFAULT true      | Account status     |
| created_at      | TIMESTAMPTZ  | DEFAULT now()     | Registration time  |
| updated_at      | TIMESTAMPTZ  | DEFAULT now()     | Last update time   |

### 4.3 sessions Table

Tracks individual visitor browsing sessions. Linked to a merchant via foreign key. Sessions have a configurable timeout (default 30 minutes of inactivity).

| Column      | Type        | Constraints        | Description            |
|-------------|-------------|--------------------|------------------------|
| id          | UUID        | PK, default uuid4  | Session ID             |
| merchant_id | UUID        | FK -> merchants.id | Owning merchant        |
| visitor_id  | VARCHAR(64) | NOT NULL           | Anonymous visitor hash |
| started_at  | TIMESTAMPTZ | DEFAULT now()      | Session start          |
| ended_at    | TIMESTAMPTZ | NULLABLE           | Session end            |
| page_count  | INTEGER     | DEFAULT 0          | Pages visited          |
| device_type | VARCHAR(20) | NULLABLE           | desktop/mobile/tablet  |
| user_agent  | TEXT        | NULLABLE           | Browser user agent     |

## 4.4 events Table

High-volume table storing every behavioral event captured by the SDK. Events are append-only and never updated. Indexed on session\_id and event\_type for fast querying.

| Column           | Type         | Constraints       | Description             |
|------------------|--------------|-------------------|-------------------------|
| id               | UUID         | PK, default uuid4 | Event ID                |
| session_id       | UUID         | FK -> sessions.id | Parent session          |
| event_type       | VARCHAR(50)  | NOT NULL, INDEXED | click/scroll/mouse/page |
| timestamp        | TIMESTAMPTZ  | NOT NULL          | Event occurrence time   |
| page_url         | TEXT         | NOT NULL          | Current page URL        |
| element_selector | VARCHAR(255) | NULLABLE          | CSS selector target     |
| payload          | JSONB        | DEFAULT '{}'      | Event-specific data     |
| x_position       | INTEGER      | NULLABLE          | Mouse X coordinate      |
| y_position       | INTEGER      | NULLABLE          | Mouse Y coordinate      |

## 4.5 session\_features Table

Stores computed ML features for each session. Populated by the feature extraction pipeline after sufficient events are collected. One row per session.

| Column             | Type        | Constraints           | Description              |
|--------------------|-------------|-----------------------|--------------------------|
| id                 | UUID        | PK, default uuid4     | Feature row ID           |
| session_id         | UUID        | FK -> sessions.id, UQ | Linked session           |
| hesitation_score   | FLOAT       | DEFAULT 0             | Mouse hesitation metric  |
| rage_click_count   | INTEGER     | DEFAULT 0             | Rapid re-clicks detected |
| scroll_retreat_pct | FLOAT       | DEFAULT 0             | Backward scroll ratio    |
| exit_intent_count  | INTEGER     | DEFAULT 0             | Cursor-to-close events   |
| intent_score       | FLOAT       | NULLABLE              | ML purchase intent (0-1) |
| friction_score     | FLOAT       | NULLABLE              | ML friction level (0-1)  |
| computed_at        | TIMESTAMPTZ | DEFAULT now()         | Computation timestamp    |

## 4.6 experiments Table

Manages A/B testing experiments. Each experiment defines a control and variant configuration, target audience percentage, and tracks status lifecycle.

| Column         | Type         | Constraints        | Description                |
|----------------|--------------|--------------------|----------------------------|
| id             | UUID         | PK, default uuid4  | Experiment ID              |
| merchant_id    | UUID         | FK -> merchants.id | Owning merchant            |
| name           | VARCHAR(255) | NOT NULL           | Experiment name            |
| description    | TEXT         | NULLABLE           | Experiment description     |
| status         | VARCHAR(20)  | DEFAULT 'draft'    | draft/running/paused/done  |
| variant_config | JSONB        | NOT NULL           | Control + variant setup    |
| traffic_pct    | FLOAT        | DEFAULT 0.5        | Variant traffic allocation |
| started_at     | TIMESTAMPTZ  | NULLABLE           | Experiment start           |
| ended_at       | TIMESTAMPTZ  | NULLABLE           | Experiment end             |
| created_at     | TIMESTAMPTZ  | DEFAULT now()      | Creation timestamp         |



## 4.7 intervention\_results Table

Records the outcome of each intervention delivered to a visitor. Links back to the session and optionally to an experiment for A/B attribution.

| Column            | Type        | Constraints          | Description           |
|-------------------|-------------|----------------------|-----------------------|
| id                | UUID        | PK, default uuid4    | Result ID             |
| session_id        | UUID        | FK -> sessions.id    | Target session        |
| experiment_id     | UUID        | FK -> experiments.id | Linked experiment     |
| intervention_type | VARCHAR(50) | NOT NULL             | discount/urgency/exit |
| shown_at          | TIMESTAMPTZ | DEFAULT now()        | When displayed        |
| clicked           | BOOLEAN     | DEFAULT false        | User clicked CTA      |
| converted         | BOOLEAN     | DEFAULT false        | Led to purchase       |
| revenue_delta     | FLOAT       | DEFAULT 0            | Revenue impact (\$)   |

### Schema Design Decisions

- UUID primary keys: Enable distributed ID generation without coordination.
- JSONB for payload/config: Flexible schema for event data and experiment variants.
- TIMESTAMPTZ everywhere: All times stored in UTC, converted at display layer.
- RLS policies: PostgreSQL row-level security ensures tenant data isolation.

# Chapter 5

## Authentication System

---

### 5.1 Dual Authentication Model

---

Conversiono uses two distinct authentication mechanisms, each optimized for its use case. The JavaScript SDK uses API key authentication for simplicity and performance, while the merchant dashboard uses JWT bearer tokens for stateless session management.

### 5.2 SDK Key Authentication

---

The SDK authenticates requests using an X-SDK-Key HTTP header. Each merchant receives a unique 64-character cryptographically random key upon registration. The key is validated on every SDK request by looking it up in the merchants table.

```
# SDK request with API key
POST /api/sdk/events
X-SDK-Key: sk_live_a1b2c3d4e5f6...
Content-Type: application/json

{"session_id": "...", "events": [...]}
```

SDK keys are designed for browser-side use and are intentionally limited in scope. They can only access SDK endpoints (event ingestion, session creation) and cannot read dashboard data or modify merchant settings.

### 5.3 JWT Bearer Token Authentication

---

Dashboard access uses JSON Web Tokens (JWT) with the HS256 algorithm. Tokens are issued upon successful login and must be included in the Authorization header for all dashboard, experiment, analytics, and merchant management endpoints.

### Token Lifecycle

1. Merchant submits email + password to POST /api/auth/login.
2. Server verifies password against bcrypt hash in database.
3. On success, server returns an access token (24h expiry) and a refresh token (7d expiry).
4. Client stores tokens and includes access token in Authorization: Bearer <token> header.
5. When access token expires, client calls POST /api/auth/refresh with the refresh token.
6. Server issues a new access token. Refresh tokens are single-use (rotated on each refresh).

```
# JWT payload structure
{
  "sub": "merchant-uuid-here",
  "email": "merchant@example.com",
  "type": "access",
  "exp": 1711929600,
  "iat": 1711843200
}
```

## 5.4 Registration Flow

New merchants register via POST /api/auth/register with email, company name, and password. The server validates the email format, checks for uniqueness, hashes the password with bcrypt (12 rounds), generates a unique SDK key, and creates the merchant record. The response includes the merchant profile and an initial access token.

## 5.5 Password Security

Passwords are hashed using bcrypt with a cost factor of 12 (approximately 250ms per hash). Passwords are never stored in plaintext and never logged. Minimum password length is enforced at 8 characters. The bcrypt algorithm includes a per-hash salt, preventing rainbow table attacks.

**Security Note**

V1 uses HS256 (symmetric) JWT signing. V2 will migrate to RS256 (asymmetric) for better key rotation and multi-service token verification.

# Chapter 6

## ML Pipeline

---

### 6.1 Pipeline Overview

---

The ML pipeline spans Epics 1 and 2 and is responsible for transforming raw behavioral events into actionable intelligence. The pipeline flows from data loading through feature extraction, model training, ensemble scoring, SHAP explainability, and finally intervention recommendation.

Pipeline Flow:

```
Raw Events -> Feature Extraction -> Model Training -> Ensemble Scoring  
-> SHAP Explanations -> Intervention Recommendations
```

### 6.2 Data Loading

---

The data\_loader module reads raw behavioral data from CSV or Parquet files. It handles data validation, type coercion, missing value imputation, and outputs a clean Pandas DataFrame ready for feature extraction. The loader supports incremental loading for large datasets and includes automatic schema validation.

### 6.3 Feature Extraction

---

The feature extraction pipeline computes behavioral signals from raw events. These features are the core of Conversiono's behavioral intelligence.

#### Hesitation Score

Measures mouse movement hesitation patterns. Computed from mouse velocity changes, pauses over interactive elements, and cursor hovering time. High hesitation

correlates with decision uncertainty and is a strong predictor of cart abandonment.

### **Rage Click Count**

Detects rapid, repeated clicks on the same or nearby elements within a short time window (typically 3+ clicks within 500ms). Rage clicks indicate user frustration with unresponsive UI elements, broken links, or confusing navigation.

### **Scroll Retreat Percentage**

Measures the ratio of backward (upward) scrolling to total scroll distance. Users who frequently scroll back up are re-reading content, comparing options, or struggling to find information. High retreat percentages suggest friction in the content layout.

### **Exit Intent Count**

Tracks cursor movements toward the browser's close/back buttons or rapid mouse movement toward the top of the viewport. Each detected exit intent gesture is counted per session. This feature directly triggers exit-intent interventions.

## **6.4 Model Training**

### **Baseline: Logistic Regression (Epic 1)**

The baseline model uses scikit-learn's LogisticRegression with L2 regularization. Two models are trained: one for purchase intent prediction (binary: will buy / won't buy) and one for friction detection (binary: experiencing friction / smooth). The baseline establishes a performance floor and serves as a sanity check for more complex models.

### **Advanced: XGBoost Ensemble (Epic 2)**

XGBoost gradient-boosted trees replace the baseline for production scoring. Key hyperparameters are tuned via cross-validation: max\_depth=6, learning\_rate=0.1, n\_estimators=200, min\_child\_weight=3. The model handles class imbalance via scale\_pos\_weight. Two separate XGBoost models are trained for intent and friction respectively.

## **6.5 Ensemble Scoring**

The ensemble module combines predictions from multiple models using weighted averaging. In V1, the ensemble weights XGBoost at 0.7 and Logistic Regression at 0.3. The ensemble outputs calibrated probability scores between 0 and 1 for both intent and friction. Ensemble scoring provides more robust predictions than any single model.

```
# Ensemble scoring
intent_score = 0.7 * xgb_intent + 0.3 * lr_intent
friction_score = 0.7 * xgb_friction + 0.3 * lr_friction

# Thresholds for intervention triggering
HIGH_INTENT = 0.7      # Likely to purchase
HIGH_FRICTION = 0.6    # Experiencing friction
EXIT_RISK = 0.8        # Likely to abandon
```

## 6.6 SHAP Explainability

SHAP (SHapley Additive exPlanations) values are computed for every prediction to explain which features drove the model's decision. This powers the dashboard's explainability panel where merchants can understand why a visitor received a specific intervention.

- Per-prediction SHAP values show feature contribution direction and magnitude
- Global SHAP summary plots rank features by overall importance
- SHAP dependence plots reveal interaction effects between features
- Force plots visualize how features push predictions above/below the baseline

## 6.7 Intervention Engine

The intervention engine maps ML scores to actionable interventions. It uses a rule-based decision tree layered on top of the ensemble scores.

- High intent + high friction -> Discount popup (reduce friction for likely buyers)
- High intent + low friction -> Urgency banner (accelerate purchase decision)
- Low intent + exit signal -> Exit-intent offer (last-chance retention)
- Low intent + low friction -> No intervention (avoid annoying casual browsers)

**ML Architecture Decision**

V1 uses batch-trained models loaded at API startup. V2 will introduce online learning with incremental model updates as new behavioral data streams in.



# Chapter 7

## JavaScript SDK

---

### 7.1 SDK Overview

---

The Conversiono JavaScript SDK (Epic 3, CONV-19 to CONV-27) is a lightweight browser library that captures user behavioral events and streams them to the backend. It is designed for minimal performance impact on merchant storefronts, with a gzipped size under 8KB and zero external dependencies.

### 7.2 Event Collectors

---

The SDK includes specialized collectors for different event types, each optimized for its data source. Collectors attach to DOM events using passive listeners to avoid blocking the main thread.

#### Click Collector

Captures click events with target element selector, coordinates, timestamp, and whether the click was on an interactive element (link, button, input). Implements rage-click detection by tracking rapid successive clicks within a 500ms window.

#### Scroll Collector

Monitors scroll position changes using `requestAnimationFrame` for throttled sampling. Records scroll direction, velocity, depth percentage, and detects scroll retreat patterns (scrolling back up after reaching a point).

#### Mouse Collector

Tracks mouse movement via `mousemove` events, heavily throttled to ~10 samples/second. Captures cursor position, velocity, and direction. Detects hesitation (slow movement

over interactive elements) and exit intent (rapid movement toward viewport top).

## 7.3 Session Management

The SDK manages visitor sessions automatically. A session is created when the SDK initializes and persists across page navigations within the same site using sessionStorage. Sessions expire after 30 minutes of inactivity. Visitor identity is maintained across sessions using a persistent anonymous ID in localStorage.

```
// SDK initialization
Conversiono.init({
  sdkKey: 'sk_live_a1b2c3d4e5f6...',
  endpoint: 'https://api.conversiono.com',
  batchSize: 20,
  flushInterval: 5000, // 5 seconds
  sessionTimeout: 1800000 // 30 minutes
});
```

## 7.4 Event Queue & Batching

Events are not sent individually but accumulated in an in-memory queue. The queue flushes to the backend when either: (a) the batch size threshold is reached (default 20 events), or (b) the flush interval timer fires (default 5 seconds), whichever comes first. The queue also flushes on page unload using the Beacon API for reliability.

- In-memory queue with configurable max size (default 500 events)
- Automatic batching reduces HTTP overhead by ~95% vs. per-event sending
- Failed batches are retried with exponential backoff (max 3 retries)
- Queue overflow drops oldest events first (FIFO eviction)

## 7.5 Transport Layer

The transport layer handles HTTP communication with the backend. It uses fetch() for normal requests and navigator.sendBeacon() for page-unload flushes. All requests include the X-SDK-Key header for authentication.

```
// Batch event payload
POST /api/sdk/events
X-SDK-Key: sk_live_...
Content-Type: application/json

{
  "session_id": "sess_abc123",
  "events": [
    {"type": "click", "timestamp": 1711843200, ...},
    {"type": "scroll", "timestamp": 1711843201, ...},
    {"type": "mouse", "timestamp": 1711843202, ...}
  ]
}
```

## 7.6 Build System

The SDK is built using Rollup with TypeScript compilation. Two output formats are produced: UMD (Universal Module Definition) for script tag inclusion and ESM (ES Modules) for bundler-based projects. Source maps are generated for debugging.

```
# Build outputs
dist/conversiono-sdk.umd.js      # Script tag usage
dist/conversiono-sdk.umd.min.js  # Minified UMD
dist/conversiono-sdk.esm.js      # ES module import
dist/conversiono-sdk.esm.min.js  # Minified ESM
```

# Chapter 8

## Dashboard Frontend

---

### 8.1 Framework & Architecture

---

The merchant dashboard (Epics 5 and 7) is built with Next.js 14 using the App Router. It leverages React Server Components where possible for improved performance, with Client Components for interactive elements. The dashboard provides a complete management interface for merchants to monitor behavior, run experiments, and view intervention results.

### 8.2 Dashboard Pages

---

#### Overview Page (/dashboard)

The landing page displays key metrics in StatCard components: total sessions, active experiments, conversion rate, and revenue impact. Below the metrics, a time-series chart shows session volume over the last 30 days. Real-time indicators pulse when new sessions arrive via WebSocket.

#### Sessions Page (/dashboard/sessions)

A paginated, filterable table of all visitor sessions. Each row shows session ID, device type, page count, duration, and ML scores (intent + friction). Clicking a session opens a detail panel with the full event timeline and SHAP explanation of the ML scores.

#### Analytics Page (/dashboard/analytics)

Cohort analytics with interactive Recharts visualizations. Includes funnel charts (visit -> engage -> intent -> convert), time-series overlays comparing cohorts, and behavioral heatmaps showing feature distributions across visitor segments.

### Experiments Page (/dashboard/experiments)

Experiment management interface with CRUD operations. Merchants can create new A/B tests, configure control/variant parameters, set traffic allocation percentages, and monitor experiment results with statistical significance indicators.

### Interventions Page (/dashboard/interventions)

Displays intervention results aggregated by type (discount, urgency, exit-intent). Shows click-through rates, conversion rates, and revenue impact for each intervention type. Includes a timeline view of recent interventions with outcome details.

### Settings Page (/dashboard/settings)

Merchant account settings: profile update (company name, email), SDK key display and regeneration, password change, GDPR data export request, and account deletion.

## 8.3 Core Components

### Sidebar

Persistent navigation sidebar with links to all dashboard pages. Highlights the active route, shows the merchant company name, and includes a logout button. Collapses to an icon-only mode on smaller screens.

### DashboardShell

Layout wrapper that provides consistent page structure: Sidebar on the left, content area on the right with a top header bar showing the page title and breadcrumbs. Handles loading states and error boundaries at the layout level.

### StatCard

Reusable metric display component with title, value, trend indicator (up/down arrow with percentage), and optional sparkline. Supports loading skeleton state and humanized number formatting (e.g., 1.2K instead of 1,234).

### Chart Components

Recharts-based visualization components wrapped for Conversiono's design system. Includes LineChart, BarChart, AreaChart, PieChart, and FunnelChart with consistent color theming, responsive sizing, and tooltip formatting.

## 8.4 Auth Guard

---

A client-side authentication guard wraps all /dashboard routes. It checks for a valid JWT token in localStorage on mount. If no token is found or the token is expired, the user is redirected to /login. The guard also handles token refresh transparently when the access token is near expiry.

## 8.5 API Client

---

A centralized Axios instance configured with the backend base URL, automatic JWT injection via request interceptor, response error handling (401 triggers logout), and TypeScript-typed response interfaces. All dashboard pages use this client via custom React hooks (useOverview, useSessions, useAnalytics, etc.).

# Chapter 9

## API Reference

### 9.1 Authentication Endpoints

| Method | Path               | Auth | Description                   |
|--------|--------------------|------|-------------------------------|
| POST   | /api/auth/register | None | Register new merchant account |
| POST   | /api/auth/login    | None | Login and receive JWT tokens  |
| POST   | /api/auth/refresh  | JWT  | Refresh access token          |
| GET    | /api/auth/me       | JWT  | Get current merchant profile  |

### 9.2 SDK Endpoints

| Method | Path              | Auth    | Description                |
|--------|-------------------|---------|----------------------------|
| POST   | /api/sdk/sessions | SDK Key | Create new visitor session |
| POST   | /api/sdk/events   | SDK Key | Ingest batch of events     |
| GET    | /api/sdk/config   | SDK Key | Get SDK runtime config     |
| POST   | /api/sdk/identify | SDK Key | Link visitor to known user |

### 9.3 Dashboard Endpoints

| Method | Path | Auth | Description |
|--------|------|------|-------------|
|--------|------|------|-------------|

|     |                              |     |                              |
|-----|------------------------------|-----|------------------------------|
| GET | /api/dashboard/overview      | JWT | Get overview stats + metrics |
| GET | /api/dashboard/sessions      | JWT | List sessions with filters   |
| GET | /api/dashboard/sessions/:id  | JWT | Get session detail + events  |
| GET | /api/dashboard/recent-events | JWT | Get recent event stream      |

## 9.4 Experiment Endpoints

| Method | Path                 | Auth | Description              |
|--------|----------------------|------|--------------------------|
| GET    | /api/experiments     | JWT  | List all experiments     |
| POST   | /api/experiments     | JWT  | Create new experiment    |
| GET    | /api/experiments/:id | JWT  | Get experiment details   |
| PUT    | /api/experiments/:id | JWT  | Update experiment config |
| DELETE | /api/experiments/:id | JWT  | Delete experiment        |

## 9.5 Intervention Endpoints

| Method | Path                       | Auth | Description                      |
|--------|----------------------------|------|----------------------------------|
| GET    | /api/interventions         | JWT  | List intervention results        |
| GET    | /api/interventions/stats   | JWT  | Get aggregate intervention stats |
| POST   | /api/interventions/trigger | JWT  | Manually trigger intervention    |

## 9.6 Analytics Endpoints

| Method | Path                   | Auth | Description                |
|--------|------------------------|------|----------------------------|
| GET    | /api/analytics/cohorts | JWT  | Get cohort analysis data   |
| GET    | /api/analytics/funnel  | JWT  | Get conversion funnel data |



## 9.7 Merchant Endpoints

| Method | Path                          | Auth | Description             |
|--------|-------------------------------|------|-------------------------|
| GET    | /api/merchants/profile        | JWT  | Get merchant profile    |
| PUT    | /api/merchants/profile        | JWT  | Update merchant profile |
| POST   | /api/merchants/sdk-key/rotate | JWT  | Rotate SDK key          |
| DELETE | /api/merchants/account        | JWT  | Delete account (GDPR)   |

## 9.8 WebSocket Endpoint

| Method | Path         | Auth | Description                    |
|--------|--------------|------|--------------------------------|
| WS     | /api/ws/live | JWT  | Real-time session event stream |

### API Versioning

All V1 endpoints are unversioned (no /v1/ prefix). V2 will introduce /v2/ prefixed endpoints while maintaining /v1/ backward compatibility.

# Chapter 10

## Security & GDPR

---

### 10.1 Security Architecture Overview

---

Security in Conversiono (Epic 6, CONV-56 to CONV-64) follows a defense-in-depth strategy with multiple layers: authentication, authorization, transport security, input validation, rate limiting, security headers, and audit logging. Each layer provides independent protection against different threat categories.

### 10.2 JWT Authentication Security

---

JWT tokens are signed with HS256 using a 256-bit secret key stored in environment variables. Access tokens expire after 24 hours and refresh tokens after 7 days. Refresh tokens are single-use and rotated on each use to prevent replay attacks. Token payloads contain minimal claims (subject, email, type, expiry) to reduce information exposure.

### 10.3 Password Hashing

---

All passwords are hashed using bcrypt with a cost factor of 12 before storage. bcrypt incorporates a per-hash random salt, making precomputed attacks infeasible. Password verification is performed in constant time to prevent timing side-channel attacks. Minimum password length of 8 characters is enforced at the API validation layer.

### 10.4 Security Headers

---

Every HTTP response includes security headers applied by the SecurityHeadersMiddleware.

- X-Content-Type-Options: nosniff - Prevents MIME type sniffing

- X-Frame-Options: DENY - Prevents clickjacking via iframe embedding
- X-XSS-Protection: 1; mode=block - Enables browser XSS filter
- Strict-Transport-Security: max-age=31536000 - Enforces HTTPS
- Content-Security-Policy: default-src 'self' - Restricts resource loading
- Referrer-Policy: strict-origin-when-cross-origin - Controls referrer leakage

## 10.5 Rate Limiting

Redis-backed sliding window rate limiting with tiered thresholds based on endpoint sensitivity.

| Endpoint Group        | Limit         | Window   | Action on Exceed    |
|-----------------------|---------------|----------|---------------------|
| Auth (login/register) | 20 requests   | 1 minute | 429 + Retry-After   |
| SDK (event ingestion) | 1000 requests | 1 minute | 429 + Retry-After   |
| Dashboard (read)      | 200 requests  | 1 minute | 429 + Retry-After   |
| Dashboard (write)     | 50 requests   | 1 minute | 429 + Retry-After   |
| Global per-IP         | 5000 requests | 1 hour   | 429 + temp IP block |

## 10.6 Audit Logging

All state-changing API operations (POST, PUT, DELETE) are logged to an audit trail. Each audit entry captures: timestamp, merchant ID, IP address, HTTP method, endpoint path, request body hash (not the body itself, for privacy), response status code, and user agent. Audit logs are retained for 90 days and can be exported for compliance review.

## 10.7 GDPR Compliance

Conversiono implements GDPR requirements across multiple dimensions.

### Consent Management

The SDK does not initialize event collection until the merchant's site has obtained visitor consent. A consent API allows the SDK to check and record consent status. No

behavioral data is captured or transmitted without explicit opt-in.

### **Data Export (Right of Access)**

Merchants can request a full export of all data associated with their account via the settings page. The export includes all sessions, events, features, experiment configurations, and intervention results in JSON format. Exports are generated asynchronously and delivered as a downloadable archive.

### **Data Deletion (Right to Erasure)**

Account deletion triggers a cascade that removes all merchant data: sessions, events, features, experiments, intervention results, and the merchant record itself. Deletion is irreversible and completed within 72 hours per GDPR requirements. Audit log entries are anonymized (merchant ID replaced with a hash) rather than deleted, to maintain security forensic capability.

# Chapter 11

## Deployment

---

### 11.1 Docker Compose Architecture

---

Conversiono deploys as a multi-container application using Docker Compose. The compose file defines four services: PostgreSQL database, Redis cache, FastAPI backend, and Next.js frontend. All services communicate over a shared Docker network.

```
# docker-compose.yml (simplified)
services:
  postgres:
    image: postgres:16-alpine
    volumes: [pgdata:/var/lib/postgresql/data]
    environment:
      POSTGRES_DB: conversiono
      POSTGRES_USER: conversiono
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    ports: ['5432:5432']

  redis:
    image: redis:7-alpine
    ports: ['6379:6379']

  backend:
    build: ./backend
    ports: ['8000:8000']
    depends_on: [postgres, redis]
    environment:
      DATABASE_URL: postgresql://...
      REDIS_URL: redis://redis:6379
      JWT_SECRET: ${JWT_SECRET}

  frontend:
    build: ./frontend
    ports: ['3000:3000']
    depends_on: [backend]
    environment:
      NEXT_PUBLIC_API_URL: http://backend:8000
```

## 11.2 Environment Variables

| Variable | Service | Required | Description |
|----------|---------|----------|-------------|
|----------|---------|----------|-------------|

|                     |          |     |                                        |
|---------------------|----------|-----|----------------------------------------|
| DATABASE_URL        | backend  | Yes | PostgreSQL connection string           |
| REDIS_URL           | backend  | Yes | Redis connection string                |
| JWT_SECRET          | backend  | Yes | 256-bit secret for JWT signing         |
| DB_PASSWORD         | postgres | Yes | Database password                      |
| CORS_ORIGINS        | backend  | No  | Allowed CORS origins (comma-separated) |
| LOG_LEVEL           | backend  | No  | Logging level (default: INFO)          |
| NEXT_PUBLIC_API_URL | frontend | Yes | Backend API base URL                   |

## 11.3 Start Script

The start.sh script provides a one-command launcher for the entire platform. It handles environment file loading, database migrations, service startup, and health checks.

```
#!/bin/bash
# start.sh - One-command platform launcher

# Load environment
source .env

# Start infrastructure
docker compose up -d postgres redis
sleep 5 # Wait for DB readiness

# Run migrations
cd backend && alembic upgrade head && cd ..

# Start application services
docker compose up -d backend frontend

# Health check
echo 'Waiting for services...'
curl --retry 10 --retry-delay 2 http://localhost:8000/health
echo 'Conversiono is running!'
echo 'Dashboard: http://localhost:3000'
echo 'API: http://localhost:8000/docs'
```

## 11.4 Alembic Migrations

Database schema changes are managed via Alembic, SQLAlchemy's migration tool. Migrations are auto-generated from ORM model changes and applied in order. Each migration includes both `upgrade()` and `downgrade()` functions for rollback support.



```
# Migration commands
alembic revision --autogenerate -m 'add experiments table'
alembic upgrade head      # Apply all pending migrations
alembic downgrade -1      # Rollback one migration
alembic history           # Show migration history
```

**Production Note**

V1 deployment uses Docker Compose suitable for single-server deployment.

V2 will introduce Kubernetes manifests for multi-node horizontal scaling.

# Chapter 12

## What V2 Looks Like

---

### 12.1 Remaining Epics Overview

---

Three epics remain for V2, building on the complete V1 foundation. These epics focus on intelligence amplification (learning from experiment history), natural language interaction (semantic search and AI-powered suggestions), and infrastructure hardening for production scale (multi-tenant SaaS with Shopify marketplace integration).

### 12.2 Epic 8: Experiment Memory & Integrations

---

Epic 8 introduces a knowledge graph that remembers outcomes from past experiments. When a merchant creates a new experiment, the system will reference historical results from similar experiments (same industry, similar visitor segments, comparable interventions) to predict likely outcomes and suggest optimal configurations.

- Experiment outcome database with embeddings for similarity search
- Cross-merchant anonymized learning (aggregate insights without data sharing)
- Automated experiment suggestions based on detected behavioral patterns
- Integration APIs for external tools (Slack notifications, webhook callbacks)
- Email campaign integration for post-visit re-engagement

### 12.3 Epic 9: Semantic Search & Suggestion Engine

---

Epic 9 adds a natural language interface to the platform. Merchants will be able to ask questions in plain English ('Why did conversions drop last week?', 'Which pages have the highest friction?') and receive AI-generated answers with supporting data.

- Vector embeddings for session behavior patterns

- Natural language query parser using LLM (Claude API integration)
- Proactive insight generation: system alerts merchants to anomalies
- Suggestion engine recommends experiments based on behavioral trends
- Conversational dashboard interface for non-technical merchants

## 12.4 Epic 10: Infrastructure Scale & Shopify

Epic 10 transforms Conversiono from a single-tenant deployment into a production-grade multi-tenant SaaS platform with Shopify marketplace presence.

- Kubernetes deployment with horizontal pod autoscaling
- Multi-tenant data isolation with schema-per-tenant PostgreSQL
- Shopify App Store listing with OAuth installation flow
- CDN-delivered SDK with edge caching for global performance
- Stripe billing integration with usage-based pricing tiers
- 99.9% SLA with multi-region failover and automated disaster recovery

### V2 Timeline

Epic 8: Estimated 3 sprints (6 weeks)

Epic 9: Estimated 3 sprints (6 weeks)

Epic 10: Estimated 4 sprints (8 weeks)

Target V2 GA: Q4 2026

# Chapter 13

## Testing

---

### 13.1 Testing Strategy

---

Conversiono employs a multi-layer testing strategy covering unit tests, integration tests, and build verification. The CI pipeline runs all test suites on every push and pull request to the main branch, blocking merges on any failure.

### 13.2 Backend Tests (104 pytest Tests)

---

The backend test suite contains 104 tests covering all API endpoints, authentication flows, middleware behavior, and error handling. Tests use pytest with async support (pytest-asyncio) and a test database that is reset between test runs.

#### Test Categories

- Auth tests: Registration, login, token refresh, invalid credentials, expired tokens (18 tests)
- SDK tests: Session creation, event ingestion, batch processing, SDK key validation (16 tests)
- Dashboard tests: Overview stats, session listing, session detail, pagination (14 tests)
- Experiment tests: CRUD operations, status transitions, validation errors (15 tests)
- Intervention tests: Result recording, aggregation, trigger logic (10 tests)
- Analytics tests: Cohort queries, funnel computation, date range filtering (8 tests)
- Merchant tests: Profile CRUD, SDK key rotation, account deletion (10 tests)
- Middleware tests: Rate limiting, security headers, audit log entries (8 tests)
- WebSocket tests: Connection, authentication, event streaming (5 tests)

## Running Backend Tests

```
cd backend
pip install -r requirements.txt
pip install pytest pytest-asyncio httpx

# Run all tests
pytest tests/ -v

# Run specific test module
pytest tests/test_experiments.py -v

# Run with coverage
pytest tests/ --cov=app --cov-report=html
```

## 13.3 Code Quality: Ruff Linting

The backend uses ruff for fast Python linting and formatting. Ruff checks are enforced in CI and must pass with zero errors before merge. The ruff configuration covers import sorting (isort-compatible), unused imports, type annotation consistency, and PEP 8 compliance.

```
# Run ruff lint
ruff check backend/

# Auto-fix fixable issues
ruff check backend/ --fix

# Format code
ruff format backend/
```

## 13.4 Frontend Build Verification

The Next.js frontend is verified by running a full production build. This catches TypeScript type errors, missing imports, invalid page configurations, and build-time rendering

failures. The build step is mandatory in CI.

```
cd frontend
npm install
npm run build    # Next.js production build
npm run lint     # ESLint checks
```

## 13.5 ML Test Suite

The ML pipeline has its own test suite verifying data loading, feature extraction correctness, model training convergence, ensemble scoring consistency, and SHAP explanation generation. Tests use synthetic datasets to ensure reproducibility.

## 13.6 CI/CD Pipeline (GitHub Actions)

The GitHub Actions workflow runs on every push to main and every pull request. It executes all test suites in parallel for fast feedback.

```
# .github/workflows/ci.yml (simplified)
name: CI
on: [push, pull_request]

jobs:
  backend:
    runs-on: ubuntu-latest
    services:
      postgres: {image: postgres:16}
      redis: {image: redis:7}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install -r backend/requirements.txt
      - run: ruff check backend/
      - run: pytest backend/tests/ -v

  frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: cd frontend && npm ci && npm run build

  ml:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install -r ml/requirements.txt
      - run: pytest ml/tests/ -v
```

**Test Coverage Goal**

V1 achieves ~85% line coverage on the backend. The V2 target is 95% with mandatory coverage thresholds enforced in CI (--cov-fail-under=95).